Reflection on Agentic AI and FDE

IS5542 Guest Lecture 2 | 17 October 2026

The most useful idea I take from Guest Lecture 2 is that building an AI application starts with deciding what result is worth delivering. A model can produce working code while the product still solves the wrong problem. Eddie Chan's account of the Forward Deployed Engineer (FDE), an engineer who works closely with clients to build and deploy solutions, made this distinction concrete. His emphasis on root cause analysis connects the technical work in this course with a harder question: what would actually improve the customer's daily work?

Consider a company asking for a customer service chatbot. The immediate response might be to choose a model and connect it to company documents. Yet customers may mainly be frustrated because staff cannot retrieve order information from an old internal system. In that situation, fluent answers would do little to remove the bottleneck. This is my own example of why Eddie's sequence of diagnosis, prototyping and deployment matters. Observing the existing workflow could reveal that reliable data access is more valuable than a conversational interface. His discussion of legacy systems and access permissions also shows why a convincing demonstration can remain difficult to deploy.

Warren's argument that AI businesses increasingly deliver results gave me a way to evaluate that proposed solution. I agree with judging an application by what it helps users achieve, but the meaning of a result needs careful definition. Counting answered enquiries could reward quick but incomplete replies. For the chatbot example, I would compare the time needed to resolve an enquiry with the existing process, including staff checking and correcting the AI's output. I would also examine unresolved cases. Even an accurate answer should fail the evaluation if it exposes information the user is not authorised to see. A business outcome therefore depends on the surrounding workflow and controls as well as the model.

For my own learning, Eddie's point about understanding what is underneath the code is especially relevant. If AI generates an integration, I still need enough technical knowledge to understand its assumptions and investigate a failure. His learning approach, which combines questioning with building a small prototype, offers a practical starting point. I would use a narrowly scoped pilot to test one workflow and inspect where human intervention is still needed. That would give me evidence for deciding what to automate next, while keeping the initial commitment manageable.

The two talks leave me with a more demanding standard for an AI project. I would want to explain why the chosen problem matters, then show whether the system improves the work once verification and correction are included. A prototype that reveals a poor assumption can still be useful. It helps avoid investing further in a product that works technically but has little reason to be used.